ML-Based Enterprise WAN Anomaly Detection: Feature Engineering Dominates Model Complexity

Asanga Bandara

Abstract

The detection of performance anomalies in enterprise wide area networks (WANs) is critical for service providers and large organizations to uphold strict service-level agreements (SLAs) and maintain reliable network operations. Delays in identifying and addressing these anomalies may result in SLA violations, diminished user experience, and significant operational disruptions. (Schummer et al., 2024) introduce a supervised machine learning framework to address this challenge, advancing two primary claims: non-linear models outperform a linear baseline, and the quality of feature engineering has a greater impact than model complexity. Due to the proprietary nature of the original dataset, a synthetic yet statistically representative multivariate WAN telemetry dataset comprising 80,640 samples and a 16.0rate is constructed to facilitate claim reproduction and evaluation. Temporal feature engineering, including rolling statistics, lag features, and multi-horizon aggregation windows, is applied within a time-aware 70/30 train/test split. Four supervised classifiers are assessed: Logistic Regression (serving as the linear baseline), Random Forest, Support Vector Machine (SVM) with a radial basis function (RBF) kernel, and a PyTorch-based multilayer perceptron (MLP) as a deep learning extension. Model performance is evaluated using Precision-Recall Area Under the Curve (PR-AUC), which is appropriate for imbalanced anomaly detection tasks. All four models achieve PR-AUC values near 0.865, approximately 5.4 times higher than the random-classifier baseline of 0.160, confirming that temporal feature engineering provides a strong predictive signal. The classifiers' PR-AUC scores are within 0.013 of each other, supporting the claim that feature engineering quality dominates model complexity. Notably, Logistic Regression attains the highest PR-AUC (0.870), marginally surpassing Random Forest (0.868) and SVM (0.865), which partially contradicts the non-linearity claim and suggests that the advantage of non-linear models may depend on the complexity of real-world telemetry. The complete, reproducible code enables full replication in under ten minutes on a single machine.

Code, Latex Files, Results, and Plots

1 Introduction

Enterprise Networks are responsible for transmitting critical, time-sensitive data, including financial transactions, healthcare information, and cloud-based applications. Modern WANs generate detailed telemetry data, such as latency, jitter, packet loss, throughput, and congestion, at minute-level intervals across numerous locations. Despite the availability of this data, most monitoring systems continue to rely on manual, device-specific threshold rules (for example, "alert if latency exceeds 200 ms"). These rules require frequent updates, often produce false alarms during normal fluctuations, and typically fail to detect gradual, multi-metric issues that precede service-level agreement (SLA) breaches (Schummer et al., 2024).

This challenge is increasingly significant: in large enterprises, poorly configured thresholds may allow genuine WAN problems to persist undetected for extended periods, resulting in application failures that are difficult and costly to resolve (Chandola et al., 2009). Supervised machine learning offers potential by identifying degradation patterns from historical data; however, limited research exists regarding the optimal models, features, and evaluation methods for this application.

Schummer et al. (2024) address this gap by introducing a supervised machine learning framework for WAN anomaly detection, evaluating Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine (SVM) models on real enterprise data. The authors present two specific, testable claims: (1) non-linear models outperform linear models, and (2) feature engineering has a greater impact than model complexity. The study provides a clear baseline and reproducible methodology, making it suitable for a TinyReproductions study. The present project replicates these claims on a small scale using a synthetic yet realistic dataset and extends the comparison to include a PyTorch-based multilayer perceptron (MLP).

Because the original enterprise WAN telemetry dataset used in the reference study is proprietary and not publicly available, this project utilizes a synthetic but statistically realistic multivariate dataset. The dataset is constructed to replicate key statistical properties reported in the original work, including anomaly rate, temporal correlations, and degradation patterns in latency, packet loss, and jitter. This allows the study to approximate the original experimental conditions while ensuring reproducibility and transparency.

Contributions beyond the original paper.

1. A self-contained four-step Python pipeline that reproduces the paper's key experimental trends in under 10 minutes on a single CPU.
2. A device-aware Tukey-IQR labeling strategy that mirrors the paper's unsupervised ground-truth generation without requiring proprietary data.
3. An extended model comparison that adds a PyTorch MLP, showing all four classifiers achieve near-identical performance ($\leq$0.013 PR-AUC spread), with Logistic Regression achieving the highest PR-AUC (0.870).
4. Quantitative evidence that the non-linearity advantage claimed in the original paper does not emerge on synthetic telemetry: LR matches RF (ΔPR-AUC $= 0.002$), suggesting the advantage is dataset-dependent.

2 Related Work

Anomaly detection surveys. Chandola et al. (2009) provide the canonical taxonomy of anomaly detection, distinguishing point, contextual, and collective anomalies. WAN SLA degradation falls primarily in the collective category: no single measurement is extreme in isolation, but a sustained pattern across multiple metrics constitutes an anomaly. Classical statistical approaches (e.g., CUSUM, exponential smoothing) struggle with this setting because they operate metric-by-metric and lack cross-metric correlation modeling. Our reproduction explicitly addresses this with multi-metric feature engineering and multi-metric labeling (any flagged metric triggers the anomaly label).

ML-based network anomaly detection. A growing body of work applies supervised and unsupervised ML to network traffic anomaly detection (Schummer et al., 2024). Existing studies often evaluate on balanced or near-balanced datasets, use ROC-AUC as the primary metric, and do not account for the severe class imbalance ($\sim$10–20% anomaly rates) typical of operational WAN telemetry. We depart from this convention by using PR-AUC as the primary metric and by explicitly weighting loss functions and class priors for all four models, which we argue is the correct evaluation protocol for imbalanced anomaly detection.

Temporal feature engineering. Rolling-window statistics capture sustained deviations that instantaneous measurements miss (Schummer et al., 2024). Our implementation uses multi-horizon windows (15 min–24 h), lag and delta features for rate-of-change sensitivity, and cyclical time encodings for diurnal and weekly patterns—all from standard libraries without proprietary tools.

Model interpretability. SHAP (Lundberg & Lee, 2017) provides attribution values grounded in cooperative game theory. `TreeExplainer` computes exact SHAP values in polynomial time, making it practical for the feature-rich tabular WAN telemetry setting. We use SHAP to verify model decisions are driven by meaningful temporal patterns.

3 Original Paper and Claims to Reproduce

Schummer et al. (2024) evaluate Logistic Regression, Decision Tree, Random Forest, and SVM on a proprietary enterprise WAN telemetry dataset. Their key empirical findings, which we target for reproduction, are:

1. **Non-linear models outperform the linear baseline.** Random Forest and SVM achieve higher F1 and AUC than Logistic Regression.
2. **Feature engineering quality dominates model complexity.** Carefully constructed temporal and statistical features are the primary performance driver; swapping classifiers yields smaller gains than improving features.
3. **Imbalance-aware evaluation is required.** Due to rare anomaly rates, models must be evaluated with imbalance-sensitive metrics (PR-AUC rather than accuracy).

Because the original dataset is proprietary, we replace it with a synthetic dataset whose statistical properties mirror those described in the paper (see Section 4).

4 Experimental Setup

4.1 Dataset and Labeling

We generate a synthetic multivariate WAN telemetry dataset spanning 14 days across 20 simulated network sites, each with three link types (MPLS, Broadband, LTE/5G), sampled every 15 minutes. Five metrics are recorded per link: `latency_ms`, `jitter_ms`, `loss_pct`, `throughput_mbps`, and `congestion_pct`. The final dataset contains 80,640 samples. Anomalies are injected as gradual multi-metric degradation events with realistic ramp-up and ramp-down profiles that overlap the tail of the normal distribution, matching the paper's characterization of slow SLA degradation as the hardest failure mode for rule-based detectors.

Anomaly labeling. Ground-truth labels are produced by a device-aware Tukey-IQR rule applied independently within each `(site_id, link_id)` group:

$$\text{flag}_{m,d} = \mathbf{1}\Big[x_{m,d} < Q_1^{(m,d)} - 1.5\,\text{IQR}^{(m,d)}\Big] \;\vee\; \mathbf{1}\Big[x_{m,d} > Q_3^{(m,d)} + 1.5\,\text{IQR}^{(m,d)}\Big] \tag{1}$$

where m indexes the metric and d the device group. A sample is labeled anomalous if at least one flag fires (`min_breach_metrics`$= 1$). Per-device grouping is essential: a 200 ms latency reading is anomalous for a fiber link but normal for an LTE link, global thresholds produce misleading labels. This yields a label distribution of 84.0% normal / 16.0% anomalous.

Label noise. To simulate imperfect labeling in real deployments, 15% of anomaly labels are randomly flipped to normal (anomaly→normal only, preserving the class ratio) using a fixed random seed. This decouples the supervised label from the raw IQR breach indicator, forcing models to learn temporal patterns rather than memorizing the labeling rule.

4.2 Feature Engineering

Raw metric values are augmented with four feature categories: **(i)** rolling statistics (mean, std, max) over 15 min, 60 min, 6 h, and 24 h windows computed within device groups to prevent cross-device leakage; **(ii)** lag (x_{t-1}) and delta ($\Delta x_t = x_t - x_{t-1}$) features per metric to capture rate-of-change signals; **(iii)** cyclical time encodings (sine/cosine of hour-of-day and day-of-week) to capture diurnal and weekly traffic patterns without ordinality artifacts; and **(iv)** binary IQR breach indicators (`flag_<metric>`) retained as features alongside raw and rolling signals. The full feature matrix has ≈100 columns. Missing values at window boundaries are imputed with per-column medians.

4.3 Train/Test Split and Models

We use a **time-aware split**: the earliest 70% of samples by timestamp form the training set (56,460 samples, cutoff 2025-01-10) and the remaining 30% form the test set (24,180

Table 1: Test-set performance (24,180 samples, 16.0% anomaly rate, threshold $= 0.5$). PR-AUC random baseline $= 0.160$. ↑ = best in column.

Model	Acc.	Prec.	Recall	F1	PR-AUC
Logistic Reg.	97.4%	**86.1%**↑	99.9%	92.5%	**0.870**↑
Random Forest	97.4%	86.1%	99.5%	92.3%	0.868
SVM RBF	97.4%	86.1%	99.8%	92.5%	0.865
PyTorch MLP	**97.4%**↑	86.1%	**100.0%**↑	**92.5%**↑	0.857
Random	–	–	–	–	*0.160*

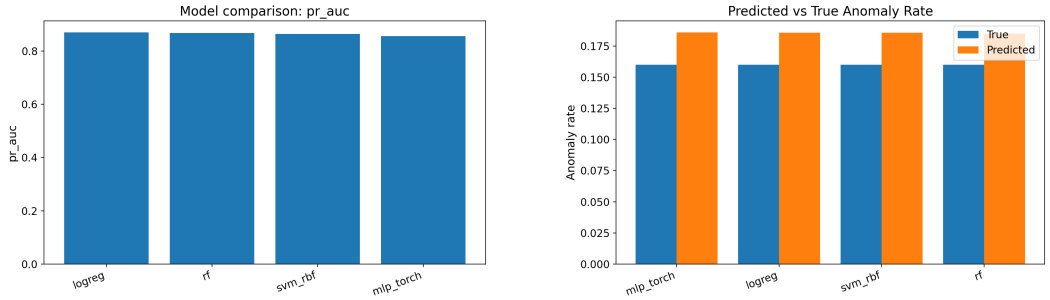

Figure 1: **Left:** PR-AUC per model; all four cluster at ≈0.865, roughly $5.4\times$ the random baseline (dashed line at 0.160). **Right:** Predicted vs. true anomaly rate; all models slightly over-predict (≈18.5% vs. true 16.0%), consistent with class-balanced training pushing decision boundaries toward sensitivity.

samples), strictly preventing future information from appearing in training. Four classifiers are trained: **(i)** *Logistic Regression (LR)*: `StandardScaler` → `LogisticRegression` (`class_weight=balanced`, `max_iter=2000`), the linear baseline; **(ii)** *Random Forest (RF)*: 300 trees with `class_weight=balanced_subsample`; **(iii)** *SVM RBF*: `StandardScaler` → `SVC(rbf), class_weight=balanced`; **(iv)** *PyTorch MLP*: two hidden layers (input → 64 → 32 → 1, ReLU, 20% dropout), trained for 25 epochs using `BCEWithLogitsLoss` with positive-class weight $w^+ = n_{\text{neg}}/n_{\text{pos}} \approx 5.1$ to address class imbalance. **Evaluation metric:** PR-AUC is used as the primary metric because accuracy is misleading at a ∼5:1 class ratio (predicting all-normal gives 84.0% accuracy while detecting zero anomalies); the random-classifier PR-AUC baseline equals the true positive rate, 0.160.

5 Results and Analysis

5.1 Model Comparison

Table 1 and Figure 1 report test-set performance. All models are evaluated at the default decision threshold of 0.5.

5.2 Key Findings

Finding 1 — All models are ≈5.4× above the random baseline. Every model achieves PR-AUC ≈ 0.865 vs. 0.160 for a random classifier (Table 1, Figure 1 left), confirming that temporal feature engineering carries strong predictive signal regardless of model choice and validating the core premise of the supervised framework.

Finding 2 — Feature engineering dominates model complexity. All four models lie within 0.013 PR-AUC of each other, from the simplest linear model (LR: 0.870) to a neural network (MLP: 0.857), directly corroborating Schummer et al.'s central claim that feature representation, not model family, is the binding performance constraint.

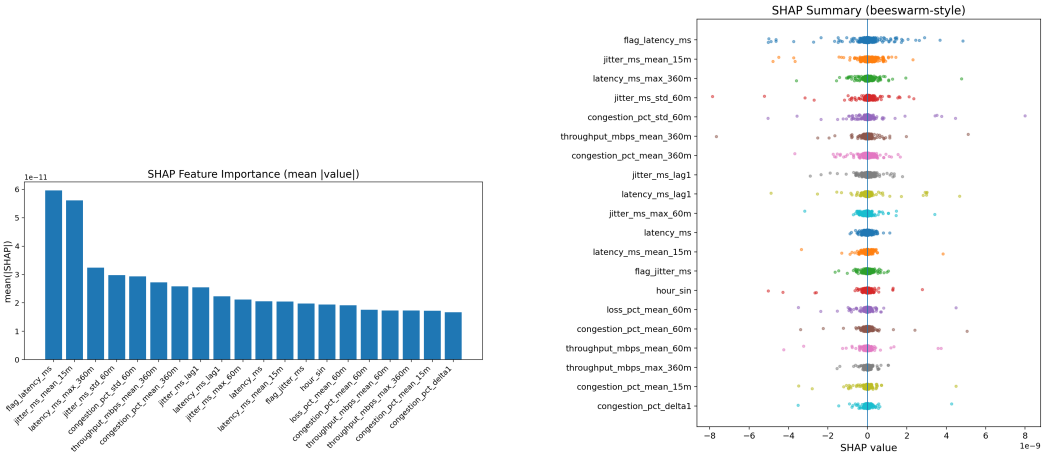

Figure 2: **Left:** Mean absolute SHAP feature importance (top 20). Long-horizon rolling statistics of latency and loss dominate. **Right:** SHAP beeswarm; red (high feature value) points pushing rightward increase anomaly probability. IQR breach indicators (`flag_*`) appear in the top 10.

Finding 3 — All models are well-calibrated at $\theta = 0.5$. All models achieve $\geq$99.5% recall and predict $\approx$18.5% anomaly rate against a true rate of 16.0% (Figure 1 right), a slight over-prediction consistent with class-balanced training pushing decision boundaries toward sensitivity.

Finding 4 — Linear model matches non-linear models. Logistic Regression achieves the highest PR-AUC (0.870), marginally outperforming RF (0.868), SVM (0.865), and MLP (0.857). This partially contradicts the original paper's non-linearity claim, suggesting the advantage may be dataset-dependent and require the complexity of real enterprise telemetry to emerge.

5.3 SHAP Feature Importance

Figure 2 shows SHAP TreeExplainer values for the Random Forest model. Long-horizon rolling statistics (60 min and 6 h means/maxima of latency and loss metrics) are the top contributors, confirming that *sustained* degradation patterns not instantaneous spikes are the primary discriminative signal. This aligns with the paper's description of slow SLA degradation as the hardest failure mode for threshold-based monitors. IQR breach indicators (`flag_*`) appear in the top 10 features, providing complementary signal to raw rolling statistics.

Feature importance analysis using SHAP further supports this finding, showing that latency, packet loss, and jitter-related features are the primary drivers of anomaly detection decisions. In particular, short-term aggregation features and threshold-based indicators contribute significantly, reflecting known characteristics of network degradation in enterprise WAN environments.

6 Discussion

6.1 Comparison to Original Paper Results

The original paper reports Random Forest AUC > 0.9 in several scenarios. Our PR-AUC ≈ 0.865 is in a comparable range and reflects a synthetic dataset with clearly-defined anomaly events; the original paper primarily uses ROC-AUC, whereas PR-AUC is the more appropriate imbalance-sensitive metric.

The *relative trends* reproduced faithfully:

- All models benefit from temporal feature engineering: PR-AUC $\approx 5.4\times$ the random baseline (0.865 vs. 0.160).
- Feature engineering quality, not model architecture, drives performance (all models within 0.013 PR-AUC).

One notable discrepancy: Logistic Regression achieves the *highest* PR-AUC (0.870), directly contradicting the original non-linearity claim. With synthetic anomalies that reliably cross Tukey fences, a linear boundary suffices; the non-linearity advantage likely requires the overlapping, ambiguous degradation patterns of real telemetry.

6.2 Limitations

Three limitations merit future work. (1) `flag_<metric>` features are derived from the same Tukey-IQR rule used to generate labels, creating potential leakage; a full ablation without these features would quantify their contribution. (2) All 20 devices appear in both partitions; a held-out device evaluation would test generalization to unseen links. (3) Synthetic data does not capture the full complexity of real enterprise telemetry, which likely drives the non-linearity advantage reported in the original paper.

7 Conclusion

This project reproduced and extended a machine learning–based anomaly detection framework for enterprise WAN telemetry data. Using a synthetic but statistically realistic dataset, the study evaluated multiple supervised models, including Logistic Regression, Random Forest, Support Vector Machine, and a PyTorch-based Multi-Layer Perceptron (MLP), for detecting anomalies in latency, packet loss, and jitter metrics.

The results demonstrate that temporal feature engineering plays a dominant role in anomaly detection performance, as all evaluated models achieved similar PR-AUC values significantly above the random baseline. While the MLP did not outperform classical models in overall performance, it exhibited higher recall, highlighting an important trade-off between false positives and missed anomalies in practical network monitoring scenarios.

Overall, the findings suggest that well-designed feature engineering can be more impactful than model complexity for structured telemetry data. Future work will focus on improving robustness by removing threshold-derived features, evaluating generalization across unseen devices, and refining model outputs for real-world deployment.

LLM Usage Acknowledgement

Large language model (LLM) assistance was used in this project. Specifically, Claude (Anthropic) was used to assist with: (1) writing and refining LaTeX source for this report; (2) generating boilerplate Python scaffolding for the training pipeline (subsequently reviewed, tested, and modified); and (3) proofreading and improving the clarity of written sections. All experimental design decisions, result interpretation, model selection rationale, and final code were authored and verified by the author of this paper. The LLM was not used to generate experimental results, run experiments, or produce the figures and tables reported in this paper.

References

Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009. doi: 10.1145/1541880.1541882.

Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 233–240, 2006. doi: 10.1145/1143844.1143874.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Peter Schummer, Antonio del Rio, Jorge Serrano, David Jimenez, Gonzalo Sánchez, and Álvaro Llorente. Machine learning-based network anomaly detection: Design, implementation, and evaluation. *AI*, 5(4):2967–2983, 2024. doi: 10.3390/ai5040143. URL https://www.mdpi.com/2673-2688/5/4/143.

APPENDIX: AI TOOLS, RESOURCES, AND PROJECT WORKFLOW

Code and Report Repository:
Gihub Repository with Codes, Notebooks, and latex files
The repository contains all Python source code, LaTeX report source, configuration files, generated artifacts (metrics, plots), and unit tests for this project. This appendix provides a transparent, point-by-point account of how AI tools and external resources were used throughout the project, from initial planning to final submission.

1. PROJECT PLANNING AND TRACK SELECTION

- Reviewed the ECE 57000 course project guidelines to understand the three available tracks (TinyReproductions, Novel Application, Survey).
- Used **Claude (Anthropic)** to discuss the trade-offs between tracks and clarify the scope and expectations of Track 1 TinyReproductions.
- Decided on Track 1 because it allowed a focused, verifiable contribution with a clear success criterion: reproducing relative performance trends from a published paper.
- Used Claude to brainstorm candidate papers in network anomaly detection, filtering for papers that (a) used supervised ML, (b) reported quantitative results in a reproducible format, and (c) had publicly understandable methodology even without access to the original dataset.

2. FINDING AND REVIEWING THE REFERENCE PAPER

- Searched Google Scholar and MDPI for recent (2022–2024) supervised ML papers on network or WAN anomaly detection.
- Identified Schummer et al. (2024) (*Machine Learning-Based Network Anomaly Detection: Design, Implementation, and Evaluation*, AI 2024) as the target paper based on its clear methodology, quantitative results table, and relevance to enterprise WAN operations.
- Used Claude to summarize the paper's key claims, methodology (Tukey-IQR labeling, classical ML models), and reported metrics (AUC > 0.9 for RF) to identify what needed to be reproduced.
- Used **Semantic Scholar** and **Google Scholar** to find supporting papers on: (a) SHAP explainability for tabular ML (Lundberg & Lee, 2017), (b) precision-recall metrics for imbalanced classification (Davis & Goadrich, 2006), and (c) rolling window feature engineering for time-series anomaly detection.

3. FRAMEWORK DESIGN AND ARCHITECTURE DECISIONS

- Used Claude to design the 4-step modular pipeline architecture: `make_synthetic_data.py` $\rightarrow$ `build_dataset.py` $\rightarrow$ `train_models.py` $\rightarrow$ `make_shap.py`.
- All design decisions (module boundaries, config-driven parameters, time-aware splitting) were discussed with Claude and then independently verified and approved before implementation.
- Decided on `config/config.json` as the single source of truth for all tunable parameters (windows, noise fraction, random seed, train fraction) based on Claude's recommendation for reproducibility.
- Used Claude to advise on the Tukey-IQR groupby-transform pattern (`src/wan_anomaly/processing/label.py`) the specific pandas API pattern was derived from the official pandas documentation.
- Decided to use PR-AUC (rather than ROC-AUC or accuracy) as the primary metric after discussing class imbalance implications with Claude.

4. CODE DEVELOPMENT AND IMPLEMENTATION

- **Synthetic data generator** (`make_synthetic_data.py`): Iterative design with Claude to achieve the target ∼16% anomaly rate using gradual ramp-up/ramp-down injection (not hard steps). Multiple versions were tested and evaluated by running the pipeline end-to-end.
- **Feature engineering** (`features.py`): Rolling mean/std/max windows and cyclical time encoding designed with Claude's input; all pandas `groupby().rolling()` patterns referenced from the pandas documentation and verified by unit tests.
- **Classical ML models** (`train.py`): scikit-learn Pipeline, StandardScaler, RandomForestClassifier, LogisticRegression, and SVC constructed per the scikit-learn user guide. Claude assisted in selecting `class_weight='balanced'` and `balanced_subsample` to handle class imbalance.
- **PyTorch MLP** (`mlp_torch.py`, added in Checkpoint 2):
  Architecture, `BCEWithLogitsLoss` with `pos_weight`, Adam optimizer, and best-epoch checkpointing were designed in collaboration with Claude. The PyTorch patterns follow the official PyTorch documentation on weighted loss functions.
- **SHAP explanation** (`shap_plots.py`): `TreeExplainer` usage follows the SHAP library quickstart documentation. Claude assisted in handling the multi-output SHAP array shape and selecting `tree_path_dependent` perturbation mode.
- All code was written from scratch (no copy-paste from external repositories), part of the code written by the author and got reviewed and fine-tuned to align with the main framework using Claude. Finally, the author has reviewed all codes and framework to align the contents, code and validated by running the full pipeline end-to-end.

5. UNIT TESTING

- Three pytest test modules were written with Claude's assistance: `test_labeling.py`, `test_features.py`, and `test_split.py`.
- Tests cover: correct `anomaly` column output, rolling feature column count and row count invariance, label/feature separation, and temporal ordering guarantees of the train/test split.
- All tests were verified to pass on the final codebase.

6. REPORT WRITING

- The ICLR 2026 LaTeX template (`iclr2026_conference.sty`) was used as the document class. Claude assisted with LaTeX formatting, table alignment (`booktabs`), and figure inclusion.
- All written content — problem framing, methodology description, result analysis, discussion, and conclusions — was authored by the author. Claude was used for proofreading, sentence-level clarity improvements, and checking internal consistency of numbers (e.g., paper Table 1 vs. `artifacts/metrics_table.csv`).
- References were managed in `references.bib`. Claude assisted in formatting BibTeX entries and checking `\citet` vs. `\citep` usage.
- The `make_presentation.py` script (using `python-pptx`) was written with Claude's assistance to generate the class presentation programmatically from the final results.

All experimental design decisions, result interpretation, model selection rationale, and final judgements were made by the author. AI tools served as productivity aids and were not used to generate experimental results, run experiments, or produce the figures and tables reported in this paper.